BÁO CÁO TỔNG HỢP VỀ SỬ DỤNG AI & HUMAN-IN-THE-LOOP

> **Môn học**: Kiểm thử phần mềm (Software Testing)
> **Sinh viên thực hiện**: 23120346
> **Đề tài**: HW02 – Domain Testing & Boundary Value Analysis on EShop
> **Tài liệu gồm 3 phần**: (1) AI Critique Report, (2) AI Gap Analysis, (3) AI Audit Report

---

BÁO CÁO PHÊ BÌNH & ĐÁNH GIÁ NĂNG LỰC AI (AI CRITIQUE REPORT)

> **Môn học**: Kiểm thử phần mềm (Software Testing)
> **Sinh viên thực hiện**: 23120346
> **Đề tài**: HW02 – Domain Testing & Boundary Value Analysis on EShop
> **Dung lượng**: ~280 từ (Đạt chuẩn 200–300 từ theo quy định đề bài)

Đoạn văn Phê bình & Rút ra Bài học (200–300 words)

Trong quá trình đồng hành cùng trợ lý AI (Antigravity LLM) thực hiện bài tập HW02, tôi nhận thấy AI thể hiện sức mạnh vượt trội ở khả năng tổng hợp lý thuyết, sinh ma trận phân hoạch tương đương (EP_V, EP_I) và xác định các điểm biên (On-point, Off-point, In-bound) rất nhanh chóng và bài bản. Tuy nhiên, AI bộc lộ những sai lệch và thiếu sót mang tính bản chất:

1. **AI sai lệch và thiếu ở đâu?** AI có thiên lệch mặc định rằng hệ thống luôn tuân thủ đặc tả (Compliance Bias). AI thiết kế test case theo "kịch bản lý tưởng" mà bỏ sót các lỗi thực tế nghiêm trọng như: thiếu hoàn toàn ô *Xác nhận mật khẩu* trên UI, lỗi bẫy bắt buộc *Double-click* mới thêm được sản phẩm vào giỏ hàng (`ProductDetail.jsx:22`), lỗi *Mass UI Overwrite* đổi tên hàng loạt sản phẩm khi sửa (`App.jsx:110`), và sự cố trình duyệt tự động điền nhầm tài khoản vào ô OTP do thiếu `autocomplete="off"` .
2. **Tại sao AI không phát hiện ra?** AI chỉ phân tích tĩnh trên văn bản đặc tả, hoàn toàn thiếu cảm nhận tương tác thực tế (Tactile Interaction) và không thể tự mình "cầm chuột nhấp" hay trải nghiệm ứng dụng như một người dùng thật. Ngoài ra, AI dạng headless không thể lường trước các cơ chế tự động điền của trình duyệt và những bẫy logic do lập trình viên cố tình cài cắm.
3. **Bài học rút ra khi phối hợp cùng AI:** AI là một công cụ hỗ trợ tuyệt vời để xây dựng khung kiểm thử (Boilerplate) và rà soát các trường hợp biên toán học; nhưng **Human-in-the-loop (Con người can thiệp)** là yếu tố sống còn không thể thay thế. Người kiểm thử phải luôn giữ tư duy phản biện (Critical Thinking), trực tiếp trải nghiệm phần mềm trên môi trường thực tế, và liên tục cung cấp ngữ cảnh mã nguồn sâu hơn để bù đắp các khoảng trống của AI.

BÁO CÁO PHÂN TÍCH KHOẢNG TRỐNG CỦA AI (AI GAP ANALYSIS)

Môn học: Kiểm thử phần mềm (HW02)

> **Sinh viên thực hiện**: 23120346
> **Mục tiêu**: So sánh kết quả kiểm thử do AI sinh ra so với quá trình rà soát thực tế của con người (Human Review), phân tích nguyên nhân tại sao AI bỏ sót.

---

1. Phân Tích Khoảng Trống Cho Tính Năng FR-03 (Quên & Đặt lại mật khẩu)

1.1. Những ca kiểm thử và Bug mà công cụ AI ban đầu bỏ sót

Khi yêu cầu AI sinh test case cho tính năng FR-03 chỉ dựa trên bản đặc tả tài liệu (Black-box Specification):

1. **AI bỏ sót lỗi thiếu trường "Xác nhận mật khẩu" (Confirm Password)**:
   - *Thực tế*: AI sinh ra rất nhiều ca kiểm thử hợp lệ kiểm tra xem `confirm_password` có khớp với `new_password` hay không. Tuy nhiên, AI mặc định rằng giao diện đã có sẵn ô nhập này và không nhận diện được rằng nhà phát triển giao diện đã **quên hoàn toàn việc tạo trường input `confirm_password` trên UI**.
2. **AI bỏ sót Bug lệch pha độ dài OTP giữa Backend và Đặc tả (4 số vs 6 số)**:
   - *Thực tế*: AI thiết kế các ca BVA với giả định OTP luôn là 6 chữ số ($N=5, 6, 7$). AI không lường trước được rằng Backend trong thực tế lại sinh mã OTP chỉ gồm 4 chữ số (`Math.random() * 9000`), và nhãn UI lại ghi là "Mã OTP (4 số)".
3. **AI bỏ sót lỗi Logic Regex mật khẩu (yêu cầu khoảng trắng thay vì ký tự đặc biệt)**:
   - *Thực tế*: AI tự động thiết kế ca kiểm thử mật khẩu hợp lệ là `Eshop@2026` và kỳ vọng hệ thống sẽ chấp nhận. Tuy nhiên, khi chạy thực tế trên code thật, hệ thống lại từ chối mật khẩu này vì lập trình viên đã viết sai regex thành `(?=.*\s)` (đòi hỏi khoảng trắng). Con người phải trực tiếp kiểm tra mã nguồn (`ForgotPassword.jsx`) hoặc test thực tế mới phát hiện ra bug logic nghiêm trọng này.
4. **AI HOÀN TOÀN BỎ SÓT LỖI TƯƠNG TÁC THỰC TẾ: Browser Autofill Misdirection (BUG-FR03-08)**:
   - *Phát hiện của Sinh viên*: Khi người dùng test trên trình duyệt thực tế có lưu tài khoản, do form thiếu thuộc tính `autocomplete="off"` , trình duyệt tự động điền địa chỉ email cá nhân (`tvquang.working@gmail.com`) vào ô "Mã OTP" và điền mật khẩu lưu sẵn vào ô "Mật khẩu mới"!
   - *Lý do AI bỏ sót*: AI chỉ chạy trong môi trường giả lập (Sandbox/Headless) không có hồ sơ người dùng thực và không có mật khẩu lưu sẵn. AI không trải nghiệm được hành vi thực tế của cơ chế Autofill Heuristics trên Chrome/Edge. Đây là minh chứng đắt giá nhất cho vai trò không thể thay thế của **Human Review** trong triết lý AI-First của bài tập này!

1.2. Phân Tích Nguyên Nhân (Root Cause Analysis)

Theo yêu cầu mục 6 của đề bài, các nguyên nhân khiến AI bỏ sót các vấn đề trên bao gồm:

1. **Giới hạn của Prompt và Phạm vi Ngữ cảnh (Context Limitations)**:
   - Prompt ban đầu thường chỉ cung cấp tài liệu đặc tả chức năng (SRS - những gì hệ thống *nên làm*). AI không được cấp toàn bộ mã nguồn của từng file giao diện React và Backend, dẫn đến việc AI thiết kế test case theo "kịch bản lý tưởng" (Ideal World) thay vì phản ánh "thực tế triển khai" (As-built SUT).
2. **Thiên lệch giả định hệ thống tuân thủ (Compliance Bias)**:
   - Các mô hình LLM có xu hướng ngầm định rằng các thành phần cơ bản của form (như trường Confirm Password) đã được cài đặt đầy đủ theo đặc tả. AI không có trực giác nghi ngờ những lỗi cơ bản mang tính "quên cài đặt" của lập trình viên.
3. **Thiếu khả năng thực thi động (Lack of Dynamic Execution)**:
   - AI dạng văn bản (LLM) chỉ thực hiện phân tích tĩnh trên lý thuyết. AI không thể trực tiếp click chuột vào giao diện, không nhìn thấy thông báo Alert bật lên, và không tương tác trực tiếp với API runtime trừ khi được kết nối vào hệ thống kiểm thử tự động chuyên sâu.

1.3. Biện Pháp Khắc Phục Của Con Người (Human Review Remediation)

- **Can thiệp thủ công**: Sinh viên trực tiếp khởi chạy SUT, mở mã nguồn `frontend-web/src/pages/ForgotPassword.jsx` và `backend/server.js` để đối chiếu từng dòng lệnh.
- **Bổ sung Test Cases**: Bổ sung các test case kiểm tra hành vi thực tế của OTP 4 số, xử lý lỗi Autofill của trình duyệt, và kiểm tra trường hợp mật khẩu có chứa khoảng trắng.
- **Đăng tải Bug Report**: Ghi nhận toàn bộ các lỗi thực tế kèm bằng chứng ảnh chụp UI và bằng chứng phân tích mã nguồn/API.

2. Phân Tích Khoảng Trống Cho Tính Năng FR-07 (Giỏ Hàng & Luồng Mua Sắm)

2.1. Bug do Con Người phát hiện mà AI hoàn toàn bỏ sót: Lỗi Bắt buộc Double-Click để Thêm vào giỏ hàng

- **Hiện tượng thực tế**: Khi người dùng vào trang chi tiết sản phẩm và bấm nút **"Thêm vào giỏ hàng"**, ở lần bấm đầu tiên hệ thống hoàn toàn **im lặng, không có bất kỳ phản hồi nào** và sản phẩm không hề được đưa vào giỏ hàng! Người dùng bắt buộc phải **nhấp đúp chuột (Double-Click)** thì sản phẩm mới được thêm vào giỏ.
- **Mã nguồn gây lỗi tại ProductDetail.jsx:22-25**:

```
const handleAddToCart = () => {
  if (clickCount === 0) {
    setClickCount(1);
    return; // Không làm gì cả ở lần đầu tiên
  }
  addToCart(product, parseInt(quantity));
  setAdded(true);
  setClickCount(0);
};
```

Lập trình viên đã cố tình cài đặt logic bẫy: nếu `clickCount === 0` thì chỉ tăng biến đếm lên 1 và `return`, bỏ qua hoàn toàn thao tác thêm vào giỏ hàng!

2.2. Tại sao công cụ AI ban đầu hoàn toàn bỏ sót Bug này?

1. **AI chỉ kiểm thử theo kịch bản lý thuyết của Đặc tả (Specification Idealism)**:
   - Bản đặc tả FR-07 và FR-06 chỉ mô tả: "*Nút Thêm vào giỏ hàng — sau khi bấm hiển thị phản hồi trực quan và thêm vào giỏ*". AI khi sinh ca kiểm thử chỉ tạo ra ca kiểm thử: "*Bấm nút Thêm vào giỏ hàng $\rightarrow$ Kỳ vọng giỏ hàng có sản phẩm*". AI ngầm định rằng sự kiện `onClick` luôn gọi hàm thêm giỏ hàng ngay lập tức.
2. **AI dạng Script/API không có cảm nhận về trải nghiệm tương tác (Tactile Interaction Absence)**:
   - Khi AI viết script tự động hóa hoặc test qua API backend (`POST /api/cart`), các hàm được kích hoạt trực tiếp từ mã lệnh và luôn trả về thành công ngay trong 1 lần gọi.
   - AI không thực sự "cầm chuột nhấp" vào màn hình như một người dùng thật, không trải qua cảm giác hụt hẫng khi bấm một nút mà màn hình không phản ứng.
3. **Giá trị cốt lõi của Human Review**:
   - Chỉ khi sinh viên trực tiếp mở trình duyệt web `http://localhost:5173`, đóng vai trò là khách hàng thực tế và click thử vào nút "Thêm vào giỏ hàng", người kiểm thử mới phát hiện ra sự bất thường: "*Tại sao tôi bấm một lần mà giỏ hàng vẫn là 0?*".
   - Đây là minh chứng rõ nét tiếp theo khẳng định: **Kiểm thử tự động hóa bằng AI chỉ kiểm tra được tính đúng đắn của logic dữ liệu (Correctness), còn trải nghiệm thao tác người dùng (Usability & User Behavior) bắt buộc phải có sự tham gia của con người (Human-in-the-loop)**.

2.3. Các Bug logic khác trong FR-07 mà AI dễ bỏ sót nếu không soi mã nguồn

1. **Bug nhân bản dòng khi thêm sản phẩm trùng (`CartContext.jsx:9`)**:
   - AI khi thiết kế test case độc lập thường chỉ kiểm thử thêm 1 sản phẩm một lần. Khi con người thao tác thực tế mua thêm chính sản phẩm đó lần 2 mới phát hiện bảng giỏ hàng sinh ra 2 dòng riêng biệt thay vì cộng dồn số lượng.
2. **Bug nút Xóa thiếu hộp thoại xác nhận (`Cart.jsx:51`)**:
   - AI thường chỉ kiểm tra tính năng xóa có làm biến mất sản phẩm khỏi giỏ hay không, dễ dàng bỏ qua việc kiểm tra xem có cửa sổ pop-up/dialog xác nhận bật lên hay không.

---

3. Phân Tích Khoảng Trống Cho Tính Năng FR-15 (Quản Lý Sản Phẩm - Admin CRUD)

3.1. Các lỗi mà AI Black-Box bỏ sót và Con Người phát hiện:

1. **Lỗi Mass UI Overwrite khi Sửa sản phẩm (`fakeMassUpdatedProducts` tại `frontend-admin/src/App.jsx:110`)**:
   - *Đặc tả*: Khi Admin bấm Sửa một sản phẩm, chỉ sản phẩm đó thay đổi thông tin.
   - *Thực tế*: Lập trình viên cố tình viết hàm `products.map(p => ({ ...p, name: productForm.name }))`, khiến cho việc sửa 1 sản phẩm làm **toàn bộ danh sách sản phẩm trên màn hình bị đổi tên theo**!
   - *Lý do AI bỏ sót*: AI phân tích từng test case độc lập ("Sửa sản phẩm X thành Y $\rightarrow$ Kỳ vọng sản phẩm X có tên Y"). AI không kiểm tra "hiệu ứng lây lan ngoài ý muốn" (Side Effects) lên các phần tử khác trong mảng state React nếu không trực tiếp nhìn thấy toàn bộ bảng danh sách sau khi submit.
2. **Lỗi API trả về HTTP 200 OK kèm `{}` khi ID không tồn tại (`server.js:161`)**:
   - *Thực tế*: Truy vấn sản phẩm không tồn tại (`GET /api/products/99999`) trả về HTTP 200 thay vì 404 Not Found, dẫn đến sập trắng trang ở Frontend.
   - *Lý do AI bỏ sót*: AI thường kiểm tra mã trạng thái HTTP theo thói quen: nếu 200 thì coi là Pass, mà không kiểm tra xem mã 200 này có phù hợp với chuẩn RESTful hay không.
3. **Lỗ hổng Broken Access Control hoàn toàn không cần Token (`server.js:167-196`)**:
   - Các API thêm/sửa/xóa sản phẩm không hề có middleware `authenticateToken`, cho phép bất kỳ ai cũng có thể xóa sạch CSDL.

---

4. Tổng Kết Giá Trị Của Human Review Trong Quy Trình AI-First

Qua quá trình thực hiện bài tập trên các tính năng thực tế, sự khác biệt giữa năng lực kiểm thử của công cụ AI và người kiểm thử con người được tổng kết như sau:

Khía Cạnh Kiểm Thử	Năng Lực Của AI	Năng Lực Của Con Người (Human Tester)
Bao phủ lý thuyết (Domain & BVA)	Cực nhanh, chính xác theo công thức toán học	Mất nhiều thời gian tính toán thủ công
Nhận diện lỗi thiếu sót UI (Missing Elements)	Thường giả định form đã có đủ trường	Nhìn bằng mắt là phát hiện ngay ô nhập bị thiếu (như Confirm Password)
Trải nghiệm tương tác (Double-click, Autofill)	Không cảm nhận được	Cảm nhận ngay sự bất thường khi bấm nút không ăn hoặc bị điền nhầm thông tin
Bẫy logic mã nguồn (Code Traps)	Bỏ sót nếu chỉ đọc đặc tả	Đọc trực tiếp code và phát hiện hàm cố tình chặn ở lần bấm đầu

Khía Cạnh Kiểm Thử	Năng Lực Của AI	Năng Lực Của Con Người (Human Tester)
Hiệu ứng phụ dữ liệu (Side Effects)	Thường kiểm tra đơn lẻ từng mục	Nhìn toàn cục bảng dữ liệu để bắt lỗi đổi tên hàng loạt

> **Bài học rút ra**: Mô hình hiệu quả nhất luôn là **AI-First with Human Review** — mình tận dụng AI để sinh nhanh khung kiểm thử và các ca biên lý thuyết, nhưng chính con người phải trực tiếp thao tác trên ứng dụng, giữ tư duy hoài nghi và phản biện để bắt được những lỗi tương tác và bẫy logic ngầm trong mã nguồn.

Phụ Lục: Nhật Ký Tương Tác AI (AI Audit Report)

> **Môn học**: Kiểm thử phần mềm (Software Testing)
> **Sinh viên thực hiện**: 23120346
> **Đề tài**: HW02 – Domain Testing & Boundary Value Analysis on EShop
> **Cam kết liêm chính học thuật**: *"Tôi sử dụng AI như một công cụ trợ lực (Co-pilot) để hỗ trợ tính toán, định dạng và dựng khung; toàn bộ việc xác thực mã nguồn, kiểm thử trên giao diện thực tế và quyết định bắt bug đều do tôi trực tiếp thực hiện."*

Danh Sách Các Lượt Tương Tác & Phản Biện Của Sinh Viên

Lượt tương tác #01: Thảo luận phương pháp & Chuẩn hóa cấu trúc bài làm

- **Thời gian**: 2026-10-16 16:20
- **Mục tiêu**: Thống nhất 4 bước Domain Testing theo đúng slide bài giảng của giảng viên, tránh việc AI sinh test case theo cảm tính hoặc sai chuẩn ISTQB.
- **Prompt của sinh viên**:

  > *"Tóm tắt yêu cầu đề bài HW02 giúp tôi. Lưu ý là quy trình Domain Testing phải tuân thủ đúng 4 bước trong slide bài giảng: 1. Identify Variables, 2. Identify Equivalence Classes, 3. Selecting Test Cases, 4. Boundary Value Analysis. Hãy lập kế hoạch chi tiết từng bước cho tôi rà soát."*

- **AI phản hồi**: Tóm tắt cấu trúc 4 bước và đề xuất kế hoạch thực hiện cho 4 tính năng đã chọn (FR-03, FR-07, FR-15, FR-04 Mobile).
- **Đánh giá & Điều chỉnh của sinh viên (Human Review)**: Tôi kiểm tra lại thấy AI đề xuất dùng kỹ thuật BVA 2 giá trị (Min, Max), tôi đã yêu cầu AI phải chuyển sang **kỹ thuật 3-value BVA** ($LB-1, LB, LB+1$ và $UB-1, UB, UB+1$) cho chặt chẽ và đạt điểm tối đa.

Lượt tương tác #02: Lập bảng phân hoạch tương đương và bắt lỗi logic FR-03 (Quên mật khẩu)

- **Thời gian**: 2026-10-17 20:25
- **Mục tiêu**: Hỗ trợ phân tích các biến đầu vào của FR-03 và rà soát bẫy mã nguồn ở Bước 2.
- **Prompt của sinh viên**:

  > *"Phân tích các biến đầu vào của tính năng FR-03 Quên mật khẩu. Tôi vừa đọc mã nguồn file ForgotPassword.jsx và thấy có điều rất lạ ở phần regex mật khẩu và mã OTP. Bạn hãy trích xuất đúng các biến trong code và lập bảng phân hoạch tương đương cho tôi."*

- **AI phản hồi**: Trích xuất các biến `email`, `otp`, `newPassword` và lập bảng EP_V, EP_I. Ban đầu AI tự ý thêm biến `confirmPassword` vào bảng.
- **Đánh giá & Điều chỉnh của sinh viên (Human Review - Bắt lỗi AI)**: Tôi lập tức chặn lại và chỉ ra lỗi suy diễn của AI: "*Trong code ForgotPassword.jsx hoàn toàn KHÔNG CÓ trường Confirm Password, lập trình viên đã quên làm trường này. Bạn không được tự bịa ra biến đó trong bảng phân hoạch thực tế*". Nhờ sự can thiệp này, bảng test case phản ánh chính xác lỗi thiếu sót thực tế của SUT (`BUG-FR03-01`).

Lượt tương tác #03: Rà soát lỗi Autofill trên trình duyệt thực tế (Human Finding)

- **Thời gian**: 2026-10-17 21:10
- **Mục tiêu**: Đưa quan sát từ trải nghiệm thực tế của con người vào báo cáo kiểm thử.
- **Prompt của sinh viên**:

> *"Khi tôi mở form Bước 2 trên Chrome bằng tài khoản cá nhân, trình duyệt tự động điền email của tôi vào ô OTP và điền mật khẩu cũ vào ô mật khẩu mới. Đây là lỗi do thiếu thuộc tính autocomplete. AI của bạn có biết lỗi này không? Hãy ghi nhận đây là lỗi do con người phát hiện trong trải nghiệm thực tế."*

- **AI phản hồi**: Xác nhận đây là lỗi `Browser Autofill Misdirection` do thuật toán heuristics của trình duyệt nhận nhầm form reset password thành form login thông thường khi thiếu `autocomplete="off"`.
- **Đánh giá & Điều chỉnh của sinh viên (Human Review)**: Tôi trực tiếp chụp ảnh màn hình lỗi trên máy tính cá nhân để làm bằng chứng xác thực (`BUG-FR03-08`) và đưa vào phần phân tích khoảng trống AI.

Lượt tương tác #04: Thẩm định tính năng FR-07 Giỏ hàng & Phát hiện bẫy Double-Click

- **Thời gian**: 2026-10-17 22:30
- **Mục tiêu**: Thiết kế bảng test case cho giỏ hàng và kiểm tra hành vi thêm sản phẩm.
- **Prompt của sinh viên**:

> *"Chuẩn bị các ca kiểm thử cho FR-07 Giỏ hàng. Khi tôi trực tiếp test tay trên web localhost:5173, tôi bấm nút Thêm vào giỏ hàng 1 lần thì không thấy có gì xảy ra, phải bấm đúp (2 lần) thì mới vào giỏ. Bạn hãy soi code ProductDetail.jsx dòng 22 xem có bẫy gì ở đây không."*

- **AI phản hồi**: Đọc mã nguồn và tìm ra đoạn code: `if (clickCount === 0) { setClickCount(1); return; }` chứng minh lập trình viên cố tình gài bẫy chặn click lần đầu.
- **Đánh giá & Điều chỉnh của sinh viên (Human Review)**: AI đề xuất log thêm một số lỗi vặt về màu sắc nút bấm, nhưng tôi kiên quyết gạt bỏ và chỉ giữ lại 2 lỗi chức năng thực sự có giá trị: Lỗi thêm trùng bị nhân đôi dòng (`BUG-FR07-02`) và Lỗi xóa không hỏi xác nhận (`BUG-FR07-03`). Bẫy double-click được tôi đưa riêng vào phần AI Gap để phân tích sự khác biệt giữa test máy và test tay.

Lượt tương tác #05: Kiểm tra biên mở rộng cho FR-15 Quản lý sản phẩm Admin

- **Thời gian**: 2026-10-17 23:15
- **Mục tiêu**: Viết script tự động quét các giá trị biên khắc nghiệt (Extreme Boundaries) trên API sản phẩm.
- **Prompt của sinh viên**:

> *"Tính năng quản lý sản phẩm này rất rộng. Tôi nghi ngờ backend chưa validate kỹ dữ liệu đầu vào. Hãy giúp tôi viết một script nodejs để gửi các payload biên: giá = 0, giá âm, giá là chữ, tên rỗng, category_id ảo không tồn tại xem backend phản ứng ra sao."*

- **AI phản hồi**: Hỗ trợ viết script `exec_fr15_extended_tests.js` để gửi các payload kiểm thử tự động lên `http://localhost:3000/api/products`.
- **Đánh giá & Điều chỉnh của sinh viên (Human Review)**: Tôi trực tiếp chạy script, quan sát mã trả về của backend và dùng trình duyệt Admin (`localhost:5174`) kiểm chứng lại. Kết quả phát hiện backend chấp nhận cả giá âm lẫn giá 0đ. Tôi đã tự tay chụp lại màn hình giao diện lưu giá 0đ thành công để làm minh chứng không thể chối cãi.

Lượt tương tác #06: Nâng cấp Expo SDK 57 và xử lý sự cố mạng phòng trọ (FR-04 Mobile)

- **Thời gian**: 2026-10-17 23:35
- **Mục tiêu**: Nâng cấp môi trường di động và thiết lập đường truyền cho điện thoại thật.
- **Prompt của sinh viên**:

> *"Ứng dụng mobile đang chạy Expo SDK cũ (54), hãy hướng dẫn tôi nâng lên SDK 57 cho tương thích với app Expo Go trên điện thoại của tôi. Mạng của tôi là mạng phòng trọ nên điện thoại không gọi được về IP 192.168.x.x của laptop, cần giải pháp tunnel công khai."*

- **AI phản hồi**: Hỗ trợ chạy `npx expo install expo@^57.0.0 --fix`, dọn dẹp các trường cấu hình cũ trong `app.json` và thiết lập tunnel HTTPS qua Pinggy cho Backend API.
- **Đánh giá & Điều chỉnh của sinh viên (Human Review)**: Tôi dùng iPhone quét mã QR, đăng nhập vào app và trực tiếp test form cập nhật hồ sơ cá nhân. Nhờ đó, tôi phát hiện ra regex cấm số 0 ở đầu (`^[1-9]`) khiến số điện thoại `0357131476` của tôi bị văng Alert từ chối. Tôi đã chụp lại màn hình điện thoại gửi vào thư mục `screenshots/`.

Lượt tương tác #07: Kiểm toán hình ảnh hiển thị trên GitHub Issues và hoàn thiện tài liệu

- **Thời gian**: 2026-10-18 00:10
- **Mục tiêu**: Đảm bảo toàn bộ 21 issues trên GitHub hiển thị ảnh Markdown trực tiếp, không bị lỗi text thô hay link 404.
- **Prompt của sinh viên**:

> *"Kiểm tra lại toàn bộ các issues trên GitHub nhóm. Tôi thấy một số issues của FR-03 đang bị hiển thị dạng text tên file chứ không hiện ảnh trực tiếp. Hãy viết script PATCH lại toàn bộ nội dung issue bằng cú pháp Markdown image tuyệt đối từ raw branch hw02-23120346."*

- **AI phản hồi**: Viết và chạy script `update_all_github_issues_images.js` để cập nhật đồng loạt các issue #39 đến #46 và #52.
- **Đánh giá & Điều chỉnh của sinh viên (Human Review)**: Tôi trực tiếp mở từng đường link Issue trên trình duyệt để kiểm tra kết quả hiển thị, xác nhận 100% hình ảnh đã hiển thị rõ ràng, chuyên nghiệp trước khi hoàn tất báo cáo.